ANOMLY, INC.

How Computation Receipts Work

Verifiable numeric computation without trusted hardware — correctness as a property of the arithmetic, not the machine

Technical overview · August 2026 · CR specification v0.1 (with v0.1.1 sampled and v0.1.2 chained receipts)

1 The foundation: exact arithmetic

Ordinary floating-point math rounds after every addition, so $(a+b)+c \neq a+(b+c)$. Every GPU, every parallelization strategy, and every math library sums in a different order — so the "same" computation produces different bits on different machines, and *"is this result correct?"* has no checkable answer. Reproducibility today means pinning one machine, one library, one execution order — and trusting whoever ran it.

Anomly computes with b-posit — a bounded profile of posit arithmetic (Gustafson & Yonemoto, 2017; Posit Standard 2022) — accumulating into a **256-bit quire**: an accumulator wide enough to hold every intermediate sum *exactly*, with no rounding until the final readout. The mathematics is published, decades-deep, and vendor-neutral by construction; what Anomly adds is the verification layer built on it. With no rounding inside the sum, accumulation order cannot matter — so **every honest execution produces identical bits: any hardware, any implementation, any parallelism**. This identity has been demonstrated between an FPGA accelerator card, x86 CPUs, and three independently written software implementations, all producing the same certificate for the same computation.

> **The core inversion:** correctness becomes a property of the *computation*, not of the machine that ran it. Two implementations agree because exact accumulation is order-independent by construction — so a mismatch means a *different computation was run*, never merely different hardware.

2 The receipt: a commitment to what was computed

The prover emits a **manifest** — a canonical JSON document that binds, by SHA-256 digest:

- the **arithmetic profile** (b-posit16, es=3, 256-bit quire — pinned to a published registry so exact arithmetic can never be silently swapped for rounded math);
- the **computation graph** — which operations, wired how;
- the **inputs**, the **model weights**, and the **output**.

The **certificate** is the hash of that canonical manifest: one string committing to everything that determines the result. Machine details — which card, which kernel, which driver — are recorded alongside but deliberately *not* certified. They are provenance for humans, never evidence, because the machine's identity is exactly what the system makes irrelevant.

3 Verification: re-execute and compare

The verifier recomputes the same computation on **their own hardware**, builds their own receipt, and compares certificates. Verification requires no trust in any vendor, enclave, or signature authority — only arithmetic.

Verdict	Meaning
ACCEPT	Certificates identical — the result is the unique correct answer to the committed computation.
REJECT	Any flipped output bit, substituted weight, or altered input produces a different digest and is caught.
MALFORMED	The receipt is internally inconsistent — e.g. it claims order-independence under a rounded arithmetic profile.
UNVERIFIABLE	The verifier cannot assess the claim (unknown profile, order-dependent arithmetic) and refuses to accept it.

The published conformance vectors include **negative vectors** — receipts a conforming verifier *must refuse*, with the required refusal. Positive vectors alone cannot establish soundness: a verifier that accepted everything would reproduce every positive digest and still be worthless.

4 Scaling: sampled receipts and receipt chains

Sampled receipts (v0.1.1). For large outputs, the verifier re-executes only *s* of *n* output rows. The sampled indices are not chosen by the prover: they derive from `sha256(manifest || challenge)`, so the prover commits to *every* row before learning which rows will be audited.

Receipt chains (v0.1.2). Streaming or layered computations emit a chain: each receipt carries its predecessor's certificate, and a closing receipt binds the chunk count and the digest of every link. Dropping, reordering, or truncating any chunk is refused. This is how a full model forward is attested: real Llama-3.2-1B weights at full width (d=2048, d_ff=8192), one receipt per layer, sixteen layers, closed — and tampering with any single layer is caught even when the cheater consistently rebuilds the entire remainder of the chain, closing receipt included.

5 The challenge protocol: beating an adaptive cheater

A cheating prover could *grind* a sampled receipt — recompute a tampered output repeatedly until the tampered rows happen to fall outside the audited sample. The defence is temporal: the challenge must not exist until after the prover has committed.

```
1. COMMIT   prover runs the computation, emits a FULL receipt
            (the output digest is now fixed)
2. CHALLENGE  a value neither party chose arrives:
            the published randomness of a drand beacon round
            pinned STRICTLY AFTER the commitment
3. REVEAL   prover emits the sampled receipt under that challenge
            (sample indices now derive from public randomness)
4. VERIFY   verifier re-derives the indices itself and re-executes
            them on hardware the prover does not control
```

A cheater must now either predict future public randomness or produce a tampered output whose unpredictably-chosen sample still reproduces bit-exactly on an independent implementation. Because the beacon round is public and

permanent, **any third party can re-fetch it years later and audit the entire transcript** — the verifier's honesty is no longer assumed, it is checkable. This protocol has been run end-to-end against physical FPGA silicon: the card committed, a separate machine issued the challenge, and the nonce-derived sample re-executed bit-exactly on a CPU — a different architecture and a different implementation of the same arithmetic.

6 Scope: what a receipt deliberately does not claim

Computation Receipts attest **result integrity** and nothing else. They do not provide confidentiality — the verifier must hold the operands to re-execute them, so secret inputs or weights are out of scope (that is the domain of confidential-computing and zero-knowledge approaches, which solve a different problem and can be composed with receipts). They do not attest machine identity — by design. And they do not attest wall-clock timing: a chain proves order of commitment, not when the work was done. Stating these boundaries precisely is what makes the ACCEPT verdict mean something.

7 Demonstrated today

- **Specification and conformance:** CR v0.1/.1.1/.1.2 with 12 published vectors, including three refusals; a falsifiable cross-implementation gate (two independent implementations, byte-identical output, 6/6).
- **On silicon:** receipts minted by a physical Xilinx U200 625-PE exact-quire GEMM array, verified bit-exactly against independent x86 re-execution — identical certificates across architectures.
- **The challenge protocol on real hardware:** commit on the card, nonce from a second machine, nonce-derived rows re-executed on the CPU quire — adaptive-prover evidence, not just fault evidence.
- **Public-randomness challenges:** drand beacon rounds pinned after commitment, with a transcript any third party can audit against the live beacon.
- **Model scale:** a real pretrained LLM's per-layer forward attested as a closed receipt chain at full width, with consistent-liar, truncation, and reorder attacks all refused.
- **Continuously gated:** every claim above is enforced by an automated validation suite (33/33 gates at this writing) that re-verifies the committed artifacts on every run.

> **In one sentence:** for the integrity of a numeric result, a Computation Receipt replaces a trust root you have to believe with one you can check — re-execution of exact arithmetic, on any machine, by anyone, forever.